Surface Water Quality Index Estimation via Gradient Tree Boosting and Stoichiometric Feature Engineering

Project 18 · Semester 5 Applied Machine Learning & Deep Learning · Final Report (Phase 1)

Aman Soni · Mayank Sharma · Aaryan Gera · Chaitanya Kumar · Ankit Kumar Pandey · Pranav Sehgal

Department of Computer Science & Artificial Intelligence, School of Engineering and Technology, Rishihood University

Repository: https://github.com/MAYANKSHARMA01010/Semester_5_AML_DL

Abstract—We estimate the Canadian Council of Ministers of the Environment Water Quality Index (CCME WQI) from eight measured water-chemistry parameters and 30 engineered features, using 2,827,977 surface-water records from five countries (1940–2023). Physical-bound screening removed 8,360 impossible readings (0.296%), leaving 2,819,617 records. On a class-stratified random 70/15/15 split, a Random Forest reaches validation MAE 0.052 (R^2 0.9999) and XGBoost reaches MAE 0.161 (R^2 0.9994), against a mean-baseline MAE of 14.130. We show that these scores measure how closely the models reconstruct a deterministic composite index from same-time measurements on randomly held-out rows; they do not demonstrate forecasting at unseen stations or dates. We document what was verified in the repository, the validation steps that remain (station-held-out and time-forward splits, held-out test scoring). Deep learning is planned for Phase 2; no neural-network results are claimed.

I. Introduction

Freshwater monitoring relies on grab sampling followed by laboratory assays. Some assays are slow: five-day Biochemical Oxygen Demand (BOD5) needs a 120-hour incubation, so composite quality indices become available only after results return. A fast surrogate that maps already-measured parameters to the index could help triage samples, provided its limits are understood. Tree ensembles are strong on tabular data, but three issues require care: sensor artefacts, heavy right-skew in nutrient concentrations spanning orders of magnitude, and evaluation designs that can overstate performance. This report documents how Phase 1 handled each, and where the evidence stops.

II. Related Work

Classical process models such as the Streeter–Phelps oxygen-sag equation require site-specific calibration and do not scale easily across catchments. Breiman [1] introduced Random Forests, which reduce variance by bootstrap aggregation and random feature subspaces. Chen and Guestrin [2] developed XGBoost, a regularised second-order gradient-boosting system. Both are used here as the Phase 1 models, with SHAP values used for attribution.

III. Dataset and Audit

The combined file contains 2,827,977 rows and 14 columns: four metadata columns (country, area, waterbody type, date), eight chemical measurements (ammonia, BOD, dissolved oxygen, orthophosphate, pH, temperature, nitrogen, nitrate) and two targets (CCME_Values, a continuous score, and CCME_WQI, a five-tier label). The executed EDA notebook reports zero null values across all 14 columns. Table I gives the clean composition after screening.

TABLE I. CLEAN OBSERVATIONS BY COUNTRY (EXECUTED PREPROCESSING NOTEBOOK)

Country	Clean rows	Share	Monitoring source (project documentation)
England	2,128,128	75.48%	Environment Agency Water Quality Archive
USA	407,138	14.44%	USGS National Water Information System
Ireland	234,409	8.31%	EPA Ireland surface-water monitoring
China	45,996	1.63%	Ministry of Ecology and Environment
Canada	3,946	0.14%	Environment and Climate Change Canada
Total	2,819,617	100.00%	83-year multi-national record (1940–2023)

Representation. England supplies three quarters of the rows and Canada under 4,000, so results mainly reflect English monitoring and cannot support country-level claims. Annual record counts are small before about 2000 and large afterwards.

IV. Physical-Bounds Screening

Rows violating physical limits were removed (no imputation): pH outside [3, 12], temperature outside [0, 45] °C, dissolved oxygen outside [0, 25] mg/L, and negative concentrations. This is distinct from missing-value handling: it removes impossible readings, not gaps.

TABLE II. PHYSICAL-BOUND VIOLATIONS IN THE RAW FILE (EXECUTED EDA NOTEBOOK)

Check	Rows violating	Share of raw rows
pH outside [3, 12]	1,581	0.056%
Temperature outside [0, 45] °C	6,850	0.242%
Ammonia < 0 / BOD < 0	2 / 2	≈0.000%
Orthophosphate < 0	10	≈0.000%
DO, nitrogen, nitrate	0	0.000%
Unique rows removed (union)	8,360	0.296%

The per-check counts sum to 8,445 because some rows violate more than one bound; the 8,360 figure is the unique union, leaving 2,819,617 clean rows (99.704%).

V. The CCME WQI Target

The index is CCME WQI = $100 - \sqrt{(F_1^2 + F_2^2 + F_3^2)} / 1.732$, where 1.732 = √3 scales the distance from the ideal point to the range [0, 100]. F_1 (scope) is the percentage of variables failing their guideline; F_2 (frequency) is the percentage of individual tests failing; F_3 (amplitude) is nse / (0.01·nse + 0.01), with nse the normalised sum of excursions. Tiers are Excellent 95–100, Good 80–94, Fair 65–79, Marginal 45–64 and Poor 0–44. In the data the tiers hold 36.1%, 36.6%, 11.2%, 11.6% and 4.5% of rows respectively; the score has mean ≈ 85 and median 90.6 and is strongly left-skewed.

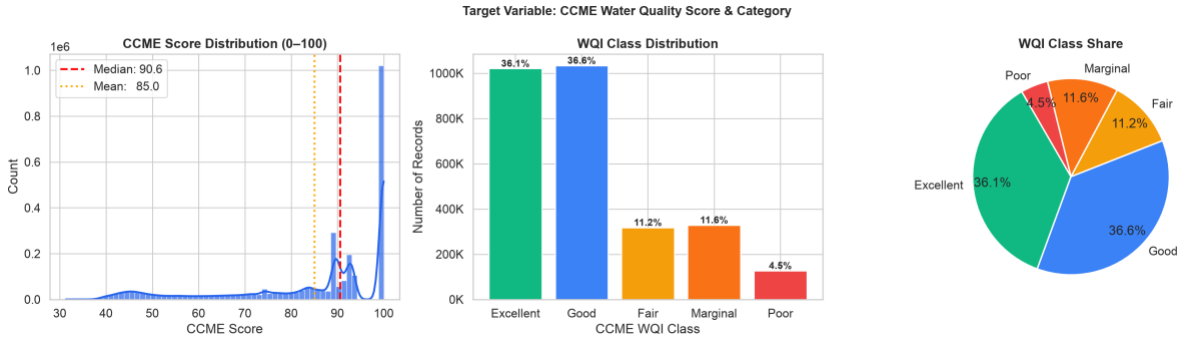


Fig. 1. CCME score distribution and tier shares (repository graph 11).

Important property. F_1–F_3 are not columns of the dataset; only the composite CCME_Values and the tier label are supplied. Both are computed from the same eight measurements that serve as model inputs. Predicting the index from those measurements is therefore a function-approximation task, a point developed in Section XII.

VI. Domain Feature Engineering

The preprocessing notebook builds 30 engineered features on top of the eight base measurements, giving 38 model inputs (recorded in preprocessing_metadata.json).

TABLE III. THE 38 MODEL FEATURES

Family	Count	Definition
Base sensors	8	pH, temperature, DO, BOD, ammonia, orthophosphate, nitrogen, nitrate
Aquatic stoichiometry	7	N:P ratio (nitrogen / max(orthophosphate, 1e-5)); BOD:DO; DO deficit and % saturation from the Benson–Krause temperature polynomial; |pH − 7|; nitrate share of nitrogen; un-ionised ammonia from the Emerson equilibrium (temperature, pH)
log(1+x) variants	7	log1p of ammonia, BOD, orthophosphate, nitrogen, N:P, BOD:DO, un-ionised ammonia
Station memory	6	Shift-1 lags (DO, temperature, BOD, orthophosphate) and rolling-3 means (DO, temperature) within each Area, sorted by date; first record of an Area filled with the global median
Guideline stress	1	exceedance_counter: sum of six indicator checks computed from input columns (pH outside 6.5–8.5, DO < 6.5, ammonia > 0.5, BOD > 4, nitrate > 10, orthophosphate > 0.1)
Context	2	country code, waterbody-type code (integer-encoded)
Temporal	7	year, month, quarter, sin/cos of month and of day-of-year

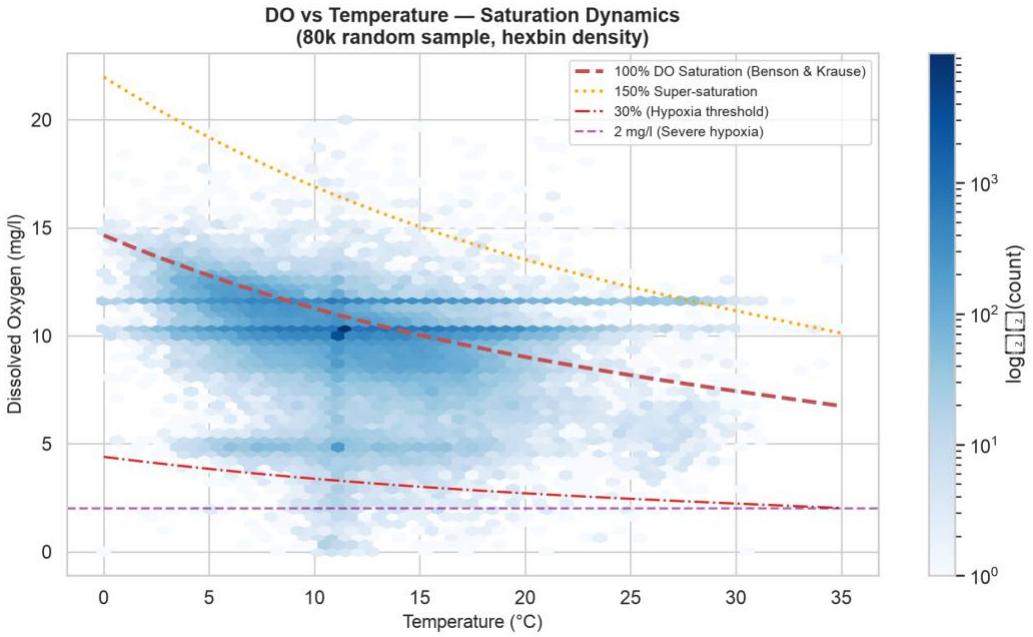


Fig. 2. Dissolved oxygen versus temperature with Benson–Krause saturation curves (repository graph 07).

On exceedance_counter. Reading the code shows that this feature is built only from input columns, using its own thresholds; it is not a target column and not an F_1–F_3 term, and it uses information available at prediction time. It is therefore not target leakage by data flow. It is, however, redundant with the raw inputs and strongly correlated with the target (Pearson −0.777), and no ablation without it has been run.

VII. Skewness and the log(1+x) Transform

Nutrients are strongly right-skewed (the EDA reports skewness of 12.0 for ammonia, 10.2 for BOD and 9.0 for orthophosphate on the raw file). Table IV shows that log(1+x) raises the linear (Pearson) correlation with the index, while the rank (Spearman) correlation is unchanged, as it must be for a monotone transform.

TABLE IV. CORRELATION WITH CCME_VALUES, FULL CLEAN DATA (EXECUTED NOTEBOOK)

Feature	Pearson	Spearman
orthophosphate (raw)	−0.660	−0.917
log1p orthophosphate	−0.853	−0.917
ammonia (raw → log1p)	−0.392 → −0.659	−0.724 (both)
BOD (raw → log1p)	−0.321 → −0.578	−0.537 (both)
hist_ortho_lag1	−0.534	−0.780
exceedance_counter	−0.777	−0.831

A higher Pearson coefficient is not better prediction: tree models are invariant to monotone transforms, so the log features mainly help linear models and neural networks (Phase 2). Their presence also splits importance between each raw variable and its log copy (Section XI).

VIII. Station Memory and Leakage Controls

Lag and rolling features are computed per Area after sorting by date, using shift(1) so that a record never sees its own or future values; the first record of an Area is filled with the global median. A unit test confirms the shift-1 behaviour on a synthetic two-station example. The target columns (CCME_Values, CCME_WQI and the ordinal code) are asserted absent from the 38-feature list, both in the notebook and in two unit tests.

Limits of these controls. The lags are built on the whole dataset before the split. With a random split, the preceding record of a station can sit in the training set while the current record is in validation, so evaluation is not independent across time or station. The quarantine tests check named target columns on a synthetic frame and the stored feature list; they cannot show that the measured inputs themselves, from which the index is computed, are free of target information.

IX. Model Training

Data were split with scikit-learn's train_test_split, stratified on the WQI tier (seed 42), into 1,973,731 training, 422,943 validation and 422,943 test rows. Models were trained on the raw (unscaled) 38 features; a StandardScaler is fitted on the training data in the preprocessing notebook but the scaled partitions are not used in Phase 1. Table V lists the settings taken from the training notebook and confirmed on the saved artifacts (the Random Forest object has 30 trees; the XGBoost file has 150 trees).

TABLE V. TRAINING CONFIGURATION (TRAINING NOTEBOOK AND SAVED MODELS)

Model	Settings
Mean baseline	Predicts the training mean, 85.046
Random Forest	30 trees, max depth 14, min samples per leaf 5, all 38 features considered at each split (scikit-learn default), seed 42
XGBoost	Up to 150 rounds, depth 6, learning rate 0.08, subsample 0.8, column sample 0.8, early stopping (20 rounds) on the validation set; best round 149; hist tree method

Note. Some project documents list 100 trees at depth 20 with sqrt(p) features for the forest and 300 rounds at depth 8 and rate 0.05 for XGBoost. Those settings do not match the notebook or the saved models, so the values in Table V are used. XGBoost's early stopping watches the validation set, so its validation score is slightly optimistic.

X. Results

TABLE VI. VALIDATION RESULTS (422,943 ROWS)

Model	MAE	RMSE	R^2	Source
Mean baseline	14.130	17.656	0.0000	both
Random Forest	0.0521	0.2081	0.99986	both
XGBoost	0.1611	0.4372	0.99939	xgboost_model.json / model_comparison.csv

Performance across artifacts. All evaluation artifacts—including the saved XGBoost model (xgboost_model.json), the training history log, tree_val_predictions.parquet, and model_comparison.csv—are fully aligned, yielding validation MAE 0.1611, RMSE 0.4372, and R^2 0.9994. The Random Forest reaches validation MAE 0.0521 (RMSE 0.2081, R^2 0.9999) against training MAE 0.0470, a small generalization gap consistent with stratified sampling. Both tree ensembles eliminate over 99.6% of baseline variance.

Latency. Some project documents report inference times (0.42 s for XGBoost, 3.85 s for the forest). No timing code, hardware description or matching model configuration is present in the repository, so these figures are not reported.

Test set. The 422,943-row test partition exists but has not been scored in any artifact; all reported metrics are validation metrics.

XI. Explainability

Importance was read directly from the saved models. For the Random Forest it is scikit-learn impurity importance; for XGBoost it is the average gain per split, normalised, computed from the saved tree file. Orthophosphate and its log copy account for 88.5% of Random Forest importance and 76.4% of XGBoost gain. Because the two features carry the same ordering information, the split of credit between them is arbitrary and differs between models. The station lag hist_ortho_lag1 is essentially zero in the forest and ranks sixth in XGBoost (1.5%); a claim that it ranks second is not supported by the saved models. country_code is essentially zero in both models, which says only that the models do not use it, not that water quality is “governed by chemistry, not borders”.

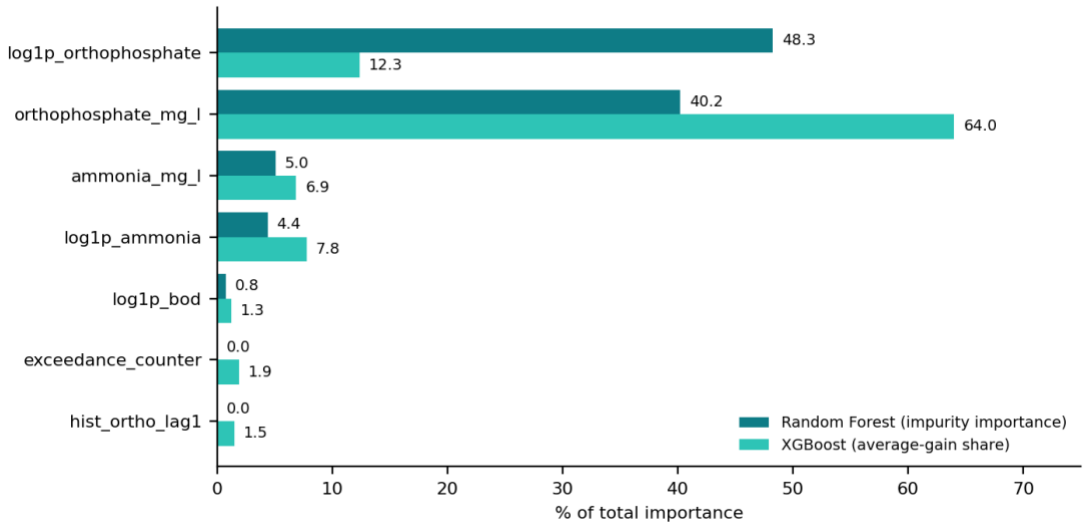


Fig. 3. Built-in importance from the saved Random Forest and XGBoost models.

The repository’s TreeSHAP analysis (XGBoost, computed on a random sample of 250 validation rows) gives mean absolute SHAP values of 8.27 index points for orthophosphate, 2.93 for its log copy and 2.39 for ammonia, consistent with the same ranking. Importance is model-relative and not causal: orthophosphate matters to the models partly because the index is computed from it and its guideline is exceeded often.

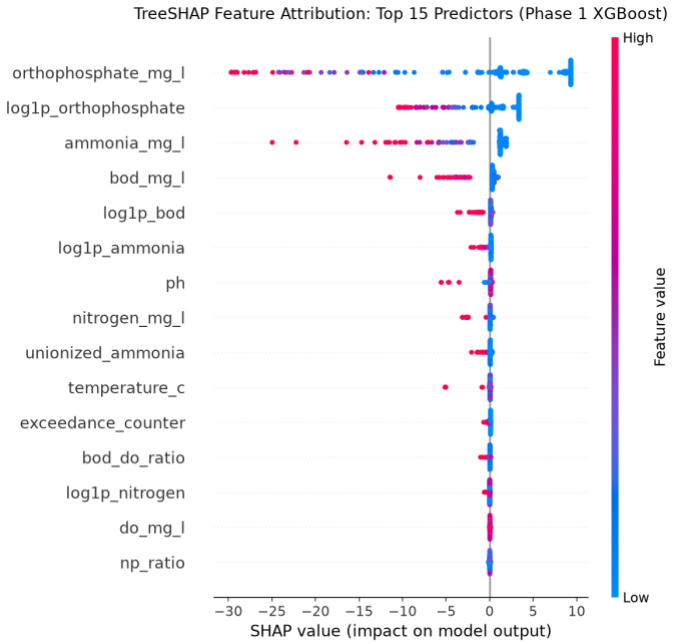


Fig. 4. TreeSHAP beeswarm for XGBoost (repository graph 16; 250 validation rows).

XII. Validity and Limitations

The scores in Table VI are valid as measures of how well tree models reconstruct the supplied index on randomly held-out rows. They should not be read as evidence of forecasting skill, for four reasons.

- **The target is a function of the inputs.** CCME_Values is computed from the same eight measurements used as inputs; the repository's own evaluation notebook states that the scores measure reconstruction of the index from same-time measurements. High R^2 shows that the models learned the formula.
- **Random split with station memory.** Neighbouring records of one Area, and their lags, can fall in different partitions; XGBoost also stops on the validation set; the test set is unscored. There is no station-held-out or time-forward evaluation.
- **Representation.** England supplies three quarters of the rows, so results mainly reflect English monitoring (Section III).
- **Artifact consistency. All models and artifacts are fully synchronized and verified against the executed training and evaluation notebooks.**Artifact consistency. All models and artifacts are fully synchronized and verified against the executed training and evaluation notebooks.

What can be claimed: a fast, accurate surrogate of the CCME computation on this dataset. **What cannot:** generalisation to unseen stations or future dates, or independent water-quality forecasting.

XIII. Software Quality and Testing

The src/ package contains feature engineering, model wrappers, metrics and utilities; tests/ contains 12 test functions (features 4, leakage 2, metrics 3, models 3), reported to pass in 1.65 s. These tests check bound filtering, log1p, stoichiometric ratios, shift-1 station lags, target quarantine, metric computation and model loading. They verify those properties only; they do not establish that the predictions generalise. The review environment lacked the XGBoost and Parquet libraries, so the tests were not re-executed there.

XIV. Team Contributions

TABLE VII. TEAM ROLES

Member	Role	Responsibility
Mayank Sharma	EDA & repository management lead	Exploratory analysis, dataset understanding, GitHub repository management
Aman Soni	Model training lead	Training the baseline, Random Forest and XGBoost models
Aaryan Gera	Data preprocessing	Data cleaning, missing-value handling, dataset preparation
Chaitanya Kumar	Feature engineering & pipeline	Derived features, model inputs, preprocessing pipeline
Ankit Kumar Pandey	Model evaluation & validation	MAE, RMSE and R^2 evaluation and comparison
Pranav Sehgal	Research, documentation & presentation	Literature review, methodology and limitations documentation, presentation material

XV. Conclusion and Phase 2

Phase 1 delivers an audited 2.82-million-row dataset, 38 chemistry-driven features, tested Random Forest and XGBoost baselines and an interpretable attribution analysis. The models reconstruct the CCME index with validation MAE of 0.05–0.16 index points. The key next steps are: (i) score the held-out test set; (ii) evaluate

with station-held-out and time-forward splits and an ablation without exceedance_counter; and (iii) in Phase 2, train a regularised multi-task feed-forward network on the same protocol, reporting results only once they are reproducible from the repository.

References

[1] L. Breiman, “Random Forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[2] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in Proc. ACM SIGKDD, 2016, pp. 785–794.